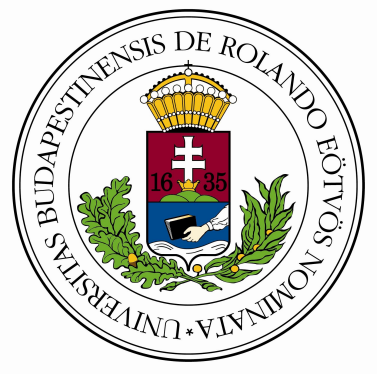
Konkurens programozás

Holtpont és elkerülése



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Több erőforrás

```
class A {
    synchronized void m1(){...}
    synchronized void m2(B b){... b.m1() ...}
}

class B {
    synchronized void m1(){...}
    synchronized void m2(A a){... a.m1() ...}
}
```



Több erőforrás

```
class A {
    synchronized void m1(){...}
    synchronized void m2(B b){... b.m1() ...}
}

class B {
    synchronized void m1(){...}
    synchronized void m2(A a){... a.m1() ...}
}

A a = new A(); B b = new B();
```

- Egyik szálban: `a.m2(b);`
- Másik szálban: `b.m2(a);`



Holtpont léphet fel!

```
class Számla {
    private int egyenleg;
    public synchronized void rátesz( int összeg ){ ... }
    public synchronized void kivesz(int összeg)
                                throws SzámlaTúllépésException {
        ...
    }
    public synchronized void átutal( int összeg, Számla másik )
                                throws SzámlaTúllépésException {
        kivesz(összeg);
        másik.rátesz(összeg);
    }
}
```



Holtpont (deadlock)

Folyamatok egy halmaza holtpontba kerül, ha a halmaz minden folyamata feltétel/időkorlát nélkül blokkolódik egy olyan erőforrásra várva, amelyet egy olyan folyamat tart lekötve, amely szintén ebben a halmazban van.



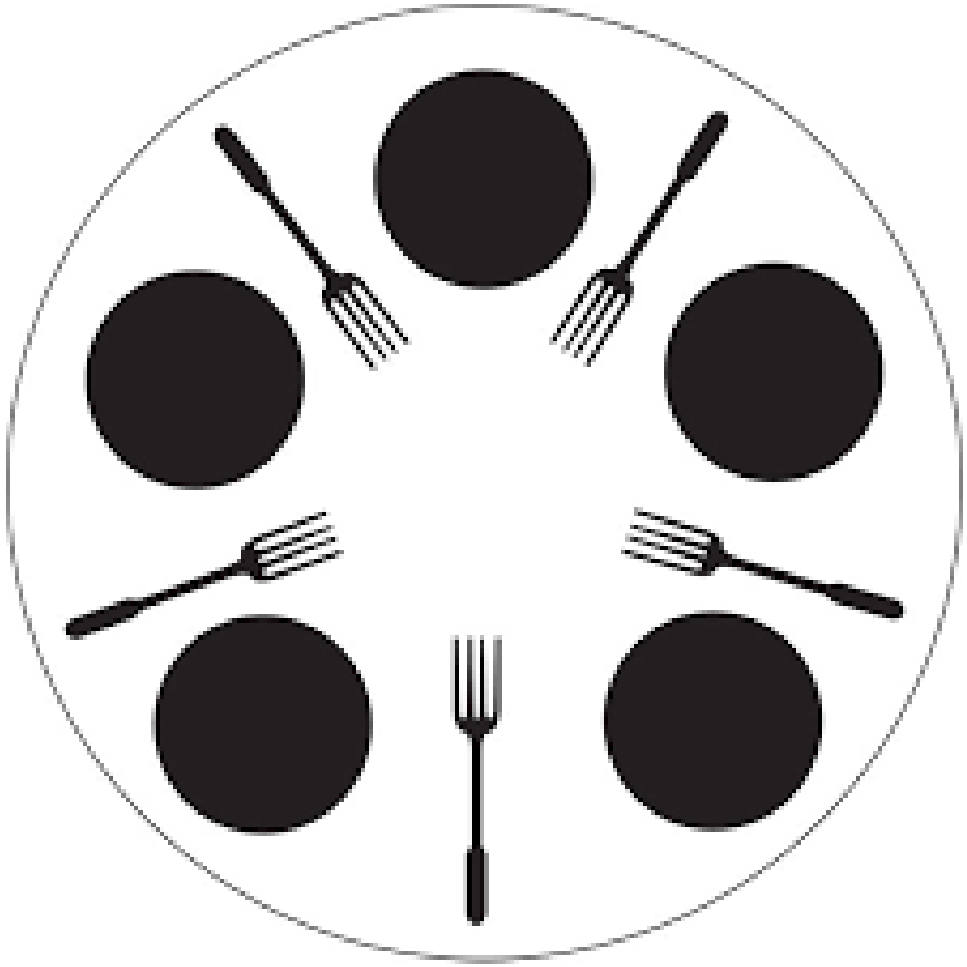
Holtpont (deadlock)

Folyamatok egy halmaza holtpontba kerül, ha a halmaz minden folyamata feltétel/időkorlát nélkül blokkolódik egy olyan erőforrásra várva, amelyet egy olyan folyamat tart lekötve, amely szintén ebben a halmazban van.

- Az interferencia tökéletes kiküszöbölése :-)
- Túl sok a szinkronizáció
- Gyakran erőforrás-kezelés vagy tranzakciók közben
- Példa: étkező filozófusok (dining philosophers)



Étkező filozófusok



Mit lehet tenni?

- Nincs univerzális megoldás, a programozó dolga a feladathoz illeszkedő megoldás kidolgozása
- Detektálás, megszüntetés (pl. timeout), előrejelzés, megelőzés
- Megelőzés: erőforrások sorrendbe állítása, szimmetria megtörése, központi irányítás, véletlenszerű várakoztatás, stb.



Csökkentsük a szinkronizációt!

```
class Számla {
    private int egyenleg;
    public synchronized int egyenleg(){ return egyenleg; }
    public synchronized void rátesz( int összeg ){ ... }
    public synchronized void kivesz(int összeg)
                                throws SzámlaTúllépésException {...}
    public void átutal( int összeg, Számla másik )
                                throws SzámlaTúllépésException {
        kivesz(összeg);
        másik.rátesz(összeg);
    }
}
```



Csökkentsük a szinkronizációt!

```
class Számla {
    private int egyenleg;
    public synchronized int egyenleg(){ return egyenleg; }
    public synchronized void rátesz( int összeg ){ ... }
    public synchronized void kivesz(int összeg)
                                throws SzámlaTúllépésException {...}
    public void átutal( int összeg, Számla másik )
                                throws SzámlaTúllépésException {
        kivesz(összeg);
        másik.rátesz(összeg);
    }
}
```

- Mások számára nehezebben érthető szinkronizációs logika
- Kód evolúciója során könnyen elronthatjuk



Bontsuk szét!

```
class Számla {
    private int egyenleg;
    public synchronized int egyenleg(){ return egyenleg; }
    public synchronized void rátesz( int összeg ){ ... }
    public synchronized void kivesz(int összeg)
                                throws SzámlaTúllépésException {...}
}

class Bank {
    public void átutal( int összeg, Számla innen, Számla ide )
                                  throws SzámlaTúllépésException {
        innen.kivesz(összeg);
        ide.rátesz(összeg);
    }
}
```



Ez sem jó: inkonzisztens vagyon

```
class Bank {
    public void átutal( int összeg, Számla innen, Számla ide )
                                    throws SzámlaTúllépésException {
        innen.kivesz(összeg);
        ide.rátesz(összeg);
    }

    public long vagyon( Számla[] számlák ){
        long összeg = 0L;
        for( Számla számla: számlák ){
            összeg += számla.egyenleg();
        }
        return összeg;
    }
}
```



Ez sem jó: túl szekvenciális az átutalás

```
class Bank {
    public synchronized void átutal( int összeg, Számla innen,
                                                 Számla ide  )
                                throws SzámlaTúllépésException {
        innen.kivesz(összeg);
        ide.rátesz(összeg);
    }
    public synchronized long vagyon( Számla[] számlák ){
        long összeg = 0L;
        for( Számla számla: számlák ){
            összeg += számla.egyenleg();
        }
        return összeg;
    }
}
```



Több erőforrás szinkronizált blokkokkal

```
class A {
    void m1(){ ... }
    void m2( B b ){ ... b.m1() ... }
}

class B {
    void m1(){ ... }
    void m2( A a ){ ... a.m1() ... }
}
```



Több erőforrás szinkronizált blokkokkal

```
class A {
    void m1(){ ... }
    void m2( B b ){ ... b.m1() ... }
}

class B {
    void m1(){ ... }
    void m2( A a ){ ... a.m1() ... }
}

A a = new A();    B b = new B();
Object lock = new Object();
```

- Egyik szálban: `synchronized( lock ){ a.m2(b); }`
- Másik szálban: `synchronized( lock ){ b.m2(a); }`



Holtpont elkerülése az erőforrások sorbarendezésével

```
public class Bank {
    ...
    public boolean átutal( int összeg, int innenId, int ideId ){
        Számla innen = számlák.get(innenId);
        Számla ide   = számlák.get(ideId);
        synchronized( innenId < ideId ? innen : ide ){
            synchronized( innenId < ideId ? ide : innen ){
                try {
                    innen.kivesz(összeg);
                    ide.rátesz(összeg);
                    return true;
                } catch( SzámlaTúllépésException e ){
                    return false;
                }
            }
        }
    }
}
```

